

TRABAJO 2 DE ROBÓTICA

PLANIFICACIÓN DE MOVIMIENTO

Juan Chauta

Ruize Wang

Índice

Primera parte	2
1. Navegación autónoma basada en comportamientos	2
2. Diseño de la misión	2
3. Comparación con métodos de planificación global.....	3
Segunda parte	4
1.Introducción	4
2.Modelado y expansión del entorno	4
3. Diagrama de visibilidad.....	5
4. Método de mapa probabilístico (Probabilistic roadmap-PRM)	8
Recomendación para BI-RRT	22

Primera parte

1. Navegación autónoma basada en comportamientos

El objetivo de esta práctica consiste en desarrollar un sistema de navegación autónoma para un robot móvil en un entorno desconocido. Para ello, se emplea una estrategia de navegación reactiva basada en comportamientos (*Behavior-Based Reactive Navigation*), donde las decisiones de movimiento se generan a partir de la información obtenida en tiempo real por los sensores del robot, sin necesidad de disponer de un mapa global del entorno ni de calcular previamente una trayectoria completa.

A diferencia de los métodos de planificación global, este enfoque permite reaccionar de forma inmediata ante cambios en el entorno y ante la aparición de obstáculos, proporcionando una solución sencilla y robusta para tareas de navegación en tiempo real.

Para alcanzar el objetivo planteado se combinan tres comportamientos fundamentales:

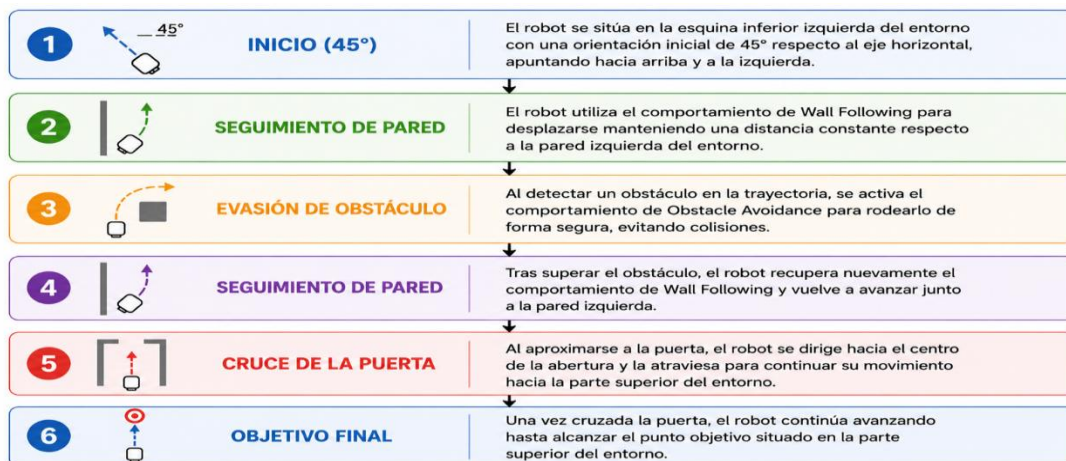
- **Wall Following:** permite mantener una distancia adecuada respecto a una pared y desplazarse siguiendo su contorno.
- **Obstacle Avoidance:** detecta obstáculos mediante los sensores y genera movimientos que evitan posibles colisiones.
- **Goal Attraction:** orienta el movimiento del robot hacia los puntos objetivo definidos durante la misión.

2. Diseño de la misión

La misión propuesta consiste en que el robot recorra el entorno siguiendo una secuencia de acciones determinada hasta alcanzar el objetivo final.

Como condición inicial, el robot se sitúa en la parte inferior izquierda del entorno con una orientación de **45° respecto al eje horizontal**, es decir, apuntando en dirección diagonal hacia arriba. Esta elección evita que el robot comience perfectamente alineado con la pared, obligándolo a corregir su orientación y distancia mediante los comportamientos de navegación implementados.

La secuencia de navegación desarrollada es la siguiente:



3. Comparación con métodos de planificación global

Aunque la navegación reactiva basada en comportamientos presenta importantes ventajas, como la simplicidad de implementación y la capacidad de reaccionar en tiempo real ante obstáculos inesperados, también posee ciertas limitaciones.

Al basarse únicamente en información local obtenida por los sensores, el robot no dispone de una visión global del entorno. Como consecuencia, las trayectorias obtenidas no siempre son óptimas y, en escenarios más complejos, pueden aparecer recorridos innecesariamente largos o situaciones de bloqueo local.

Por este motivo, en robótica también se utilizan métodos de planificación global capaces de generar trayectorias antes de iniciar el movimiento a partir de la información del entorno. Entre los algoritmos más utilizados se encuentran **Dijkstra**, **A***, **PRM**, **RRT** y **Voronoi**.

En la siguiente parte del trabajo se analizarán estos métodos de planificación de trayectorias y se estudiarán sus diferencias respecto a la navegación reactiva basada en comportamientos utilizada en esta práctica.

Segunda parte

1.Introducción

El objetivo de esta trabajo es la visualización de algunos conceptos de planificación de movimiento vistos en clase. Concretamente, el modelado del entorno su expansión, el diagrama de visibilidad, el algoritmo de Dijkstra y los métodos de PRM (probabilistic roadmap) y RRT (Rapidly exploring random trees).

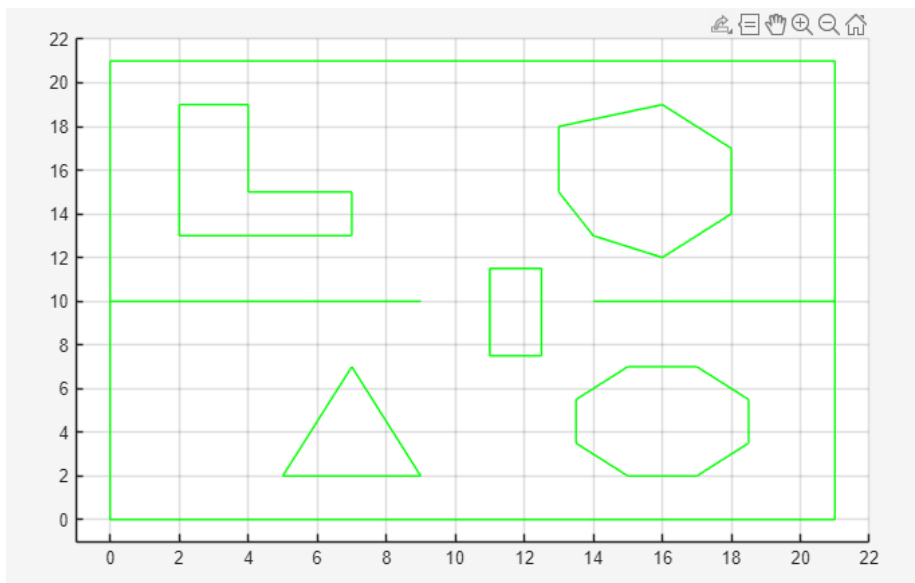
Como herramienta de simulación se utilizará el Matlab.

2.Modelado y expansión del entorno

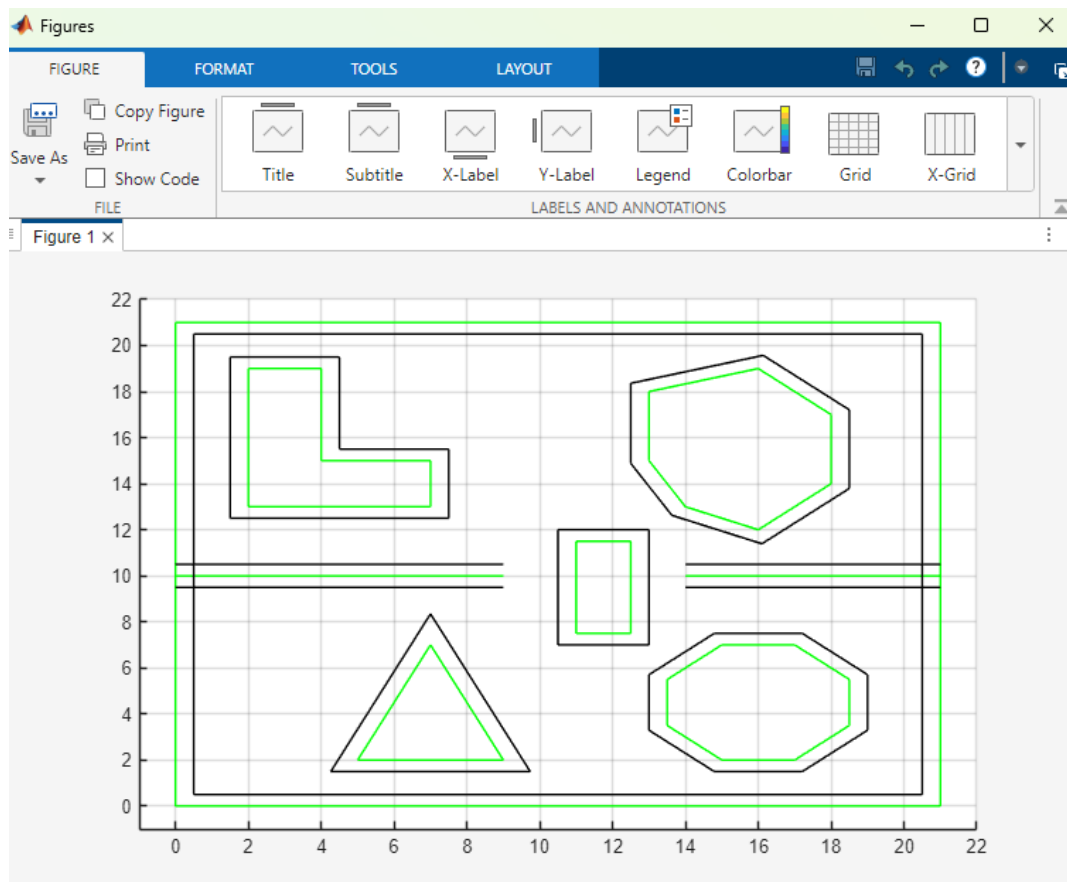
La mayoría de los algoritmos de planificación de camino están implementados para un robot puntual, sin dimensiones. Las dimensiones del entorno complican enormemente la planificación.

Por estas razones, se procede a la expansión de entorno, que consiste básicamente en el aumento del entorno original de la dimensión del robot, de forma a que no pueda impactar

Entorno



Entorno expandido

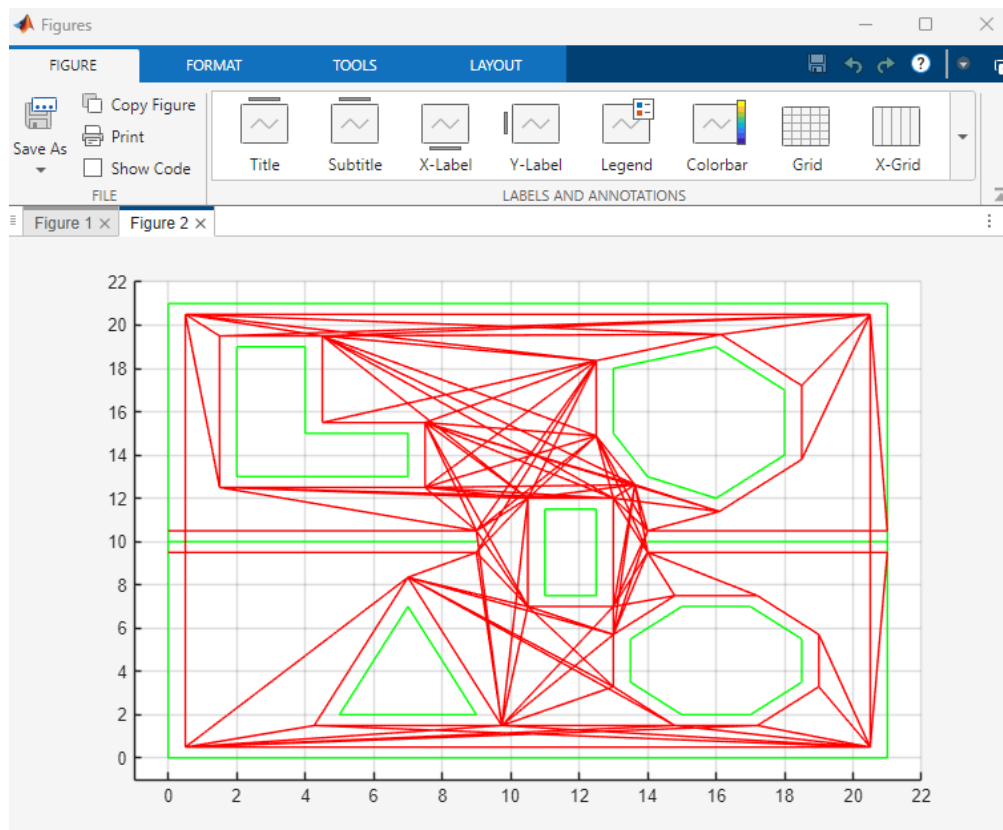


En verde está representado el entorno original, y en negro el entorno expandido. Si la planificación se hace en el expandido, el robot se podrá tratar como un punto sin dimensiones y no habrá peligro de colisiones.

Se pueden apreciar algunas cosas:

- El segundo obstáculo se ha unido con las paredes de la habitación. Se podría hacer un paso más y fusionarlos en un solo objeto. Sin embargo, esto no ha sido implementado en la versión actual dado que no es el objetivo primordial de la práctica.
- El objeto obtenido al expandir el obstáculo en forma de triángulo tiene una forma no convencional. En realidad, tiene nueve ángulos. Esta forma se ha obtenido mediante el método de expansión utilizado. Existen otros que optimizan más el espacio, pero son más complejos.
- Una vez obstáculo, la superficie dentro de los obstáculos se considerará espacio de obstáculos (Cobst) y el resto espacio libre (Cfree). De esta forma, el robot se podrá considerar como un punto sin dimensiones para la planificación de movimiento. Cabe resaltar que, en la versión actual del software, la función de expansión del entorno no funciona con objetos cóncavos ni con objetos entre los cuales la distancia es menor que la distancia a la que se expanden.

3. Diagrama de visibilidad



En rojo está representadas las líneas de visibilidad.

El siguiente paso podría ser la búsqueda del camino más corto. Para ello se ha implementado el algoritmo de Dijkstra.

Para encontrar el camino más corto, por ejemplo, entre el primer nodo y el octavo, introducir:

```
[Camino, Coste]=Dijkstra(matriz,1,8)
```

Esta función devuelve el camino más corto (secuencia de nodos por los que se debe pasar) así como su coste.

Para representar el camino gráficamente, primero se deben extraer los vértices introduciendo:

```
[vertices]=extrae_vertices(expandido)
```

```
Trayecto = dibujar_camino(Camino,vertices,[0 0 1])
```

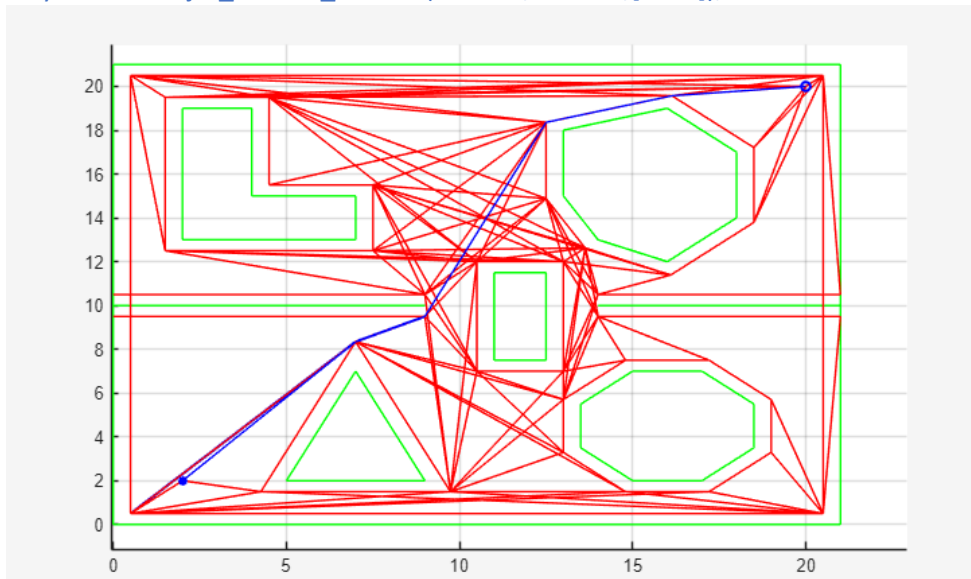
El siguiente paso podría ser la introducción del punto de partida del robot, así como del objetivo.

Para ejecutar la nueva implementación del diagrama de visibilidad debemos volver a crear el entorno original y expandido en una nueva figura y ejecutar el siguiente código:

```
ini= [2,2]; fin= [13,14]; %% Se introducen el punto  
inicial y el final.
```

```
[rectas, matriz, vertices]=rectas_visibilidad_expandido_manual(expandido, ini,fin);  
lineas=dib_rectas(rectas,[1 0 0]);
```

```
[Camino, Coste]=Dijkstra_manual(matriz);
Trayecto = dibujar_camino_manual(Camino,vertices,[0 0 1]);
```



% 1. Definir tus puntos reales de navegación Manual

```
ini = [2, 2];
```

```
fin = [20, 20];
```

% 2. Generar el entorno y expandirlo

```
ent = entorno1();
```

```
expandido = expandir_entorno(ent, 0.5);
```

% 3. USAR LA VERSIÓN MANUAL

% Esta función añade 'ini' como el primer vértice (nodo 1)

% y 'fin' como el último vértice de la lista.

```
[rectas, matriz, vertices] = rectas_visibilidad_expandido_manual(expandido, ini, fin);
```

% 4. Dibujar visibilidad

```
figure;
```

```
dibuent(ent, [0, 1, 0]); hold on;
```

```
dibuent(expandido, [0, 0, 0]);
```

```
lineas = dib_rectas(rectas, [1 0 0]);
```

% 5. Calcular Dijkstra

% Ahora, el nodo 1 es SIEMPRE tu 'ini' [2,2]

% El último nodo es SIEMPRE tu 'fin' [20,20]

```
nodo_final = size(vertices, 1);
```

```
[Camino, Coste] = Dijkstra(matriz, 1, nodo_final);
```

% 6. Dibujar el trayecto

```
Trayecto = dibujar_camino(Camino, vertices, [0 0 1]);
```

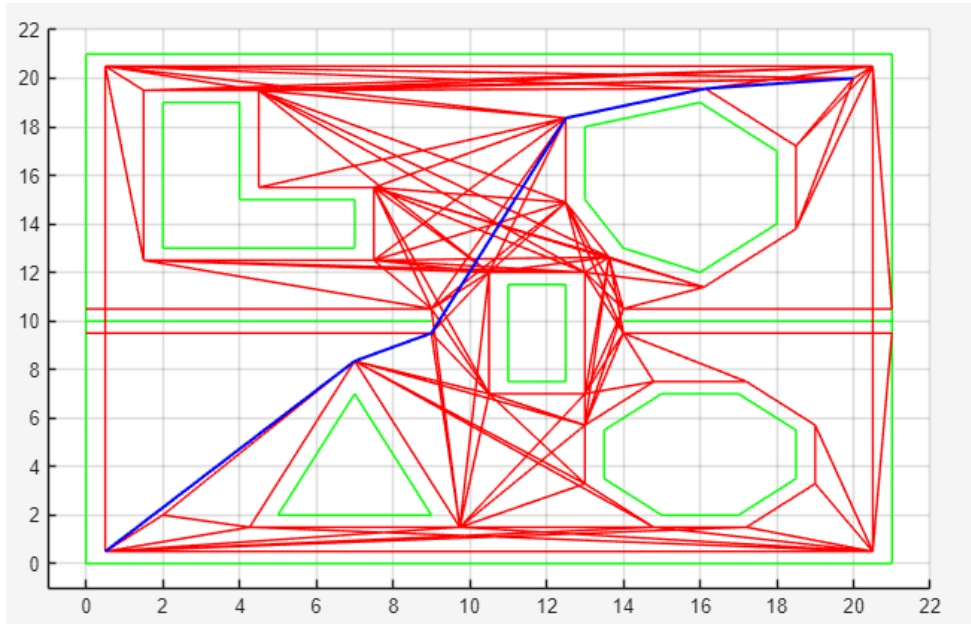
```
axis([-1 22 -1 22]); grid on;
```

En esta nueva versión del diagrama de visibilidad se permite introducir como parámetro (ini y fin) los puntos origen y destino. Al introducir estos puntos se les trata como unos nuevos vértices para poder realizar el diagrama. Para realizar esto se ha modificado la función de

“rectas_visibilidad_expandido” pasando a ser “rectas_visibilidad_expandido_manual” para poder usar como parámetros los puntos.

También se ha modificado el algoritmo de Dijkstra pasando a ser “Dijkstra_manual” en el cual ya no se le pasan los nodos origen y destino. Por último, se ha modificado la función de “dibujar_camino” pasando a ser “dibujar_camino_manual” para dibujar correctamente el camino al elegir manualmente los puntos.

El resultado es el siguiente:



- **Anotar el coste del camino para compararlo con el obtenido con otros métodos.**
Cabe resaltar que este método funciona muy bien con entornos poligonales 2D, pero peor con otros.

4. Método de mapa probabilístico (Probabilistic roadmap-PRM)

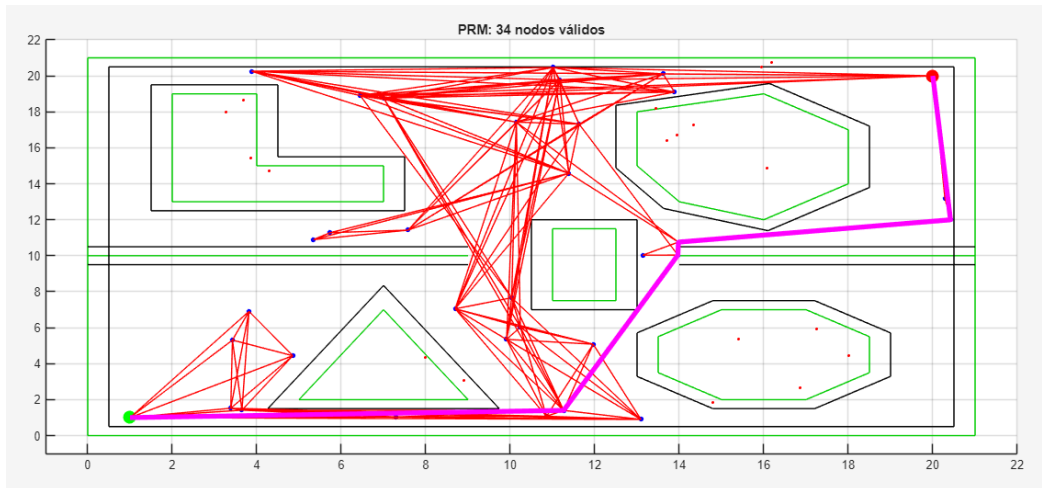
Este método de búsqueda del camino óptimo consiste en dos fases: la de aprendizaje y la de consulta. La primera consiste en la formación del mapa y, si es necesario, su expansión. Se procede a la expansión si se demuestra que el número de nodos inicial no es suficiente.

La fase de aprendizaje consta de los siguientes pasos:

1. Generar de forma aleatoria un cierto número de puntos.
2. Descartar aquellos que estén dentro de un obstáculo (Cobst).
3. Unir los restantes por arcos.
4. Descartar los arcos que crucen algún obstáculo.
5. Formar una matriz de costes con los arcos restantes.

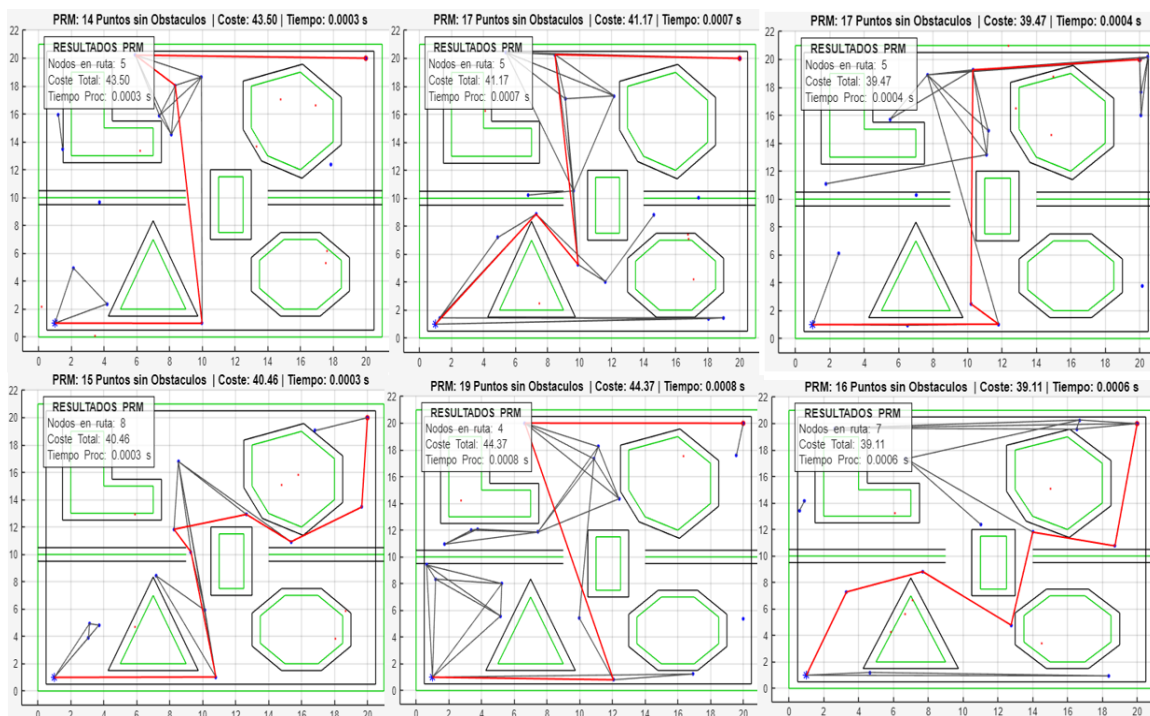
En la fase de consulta se realiza lo siguiente:

1. Se añaden los nodos de inicio y objetivo al grafo.
2. Se conectan con los demás nodos, descartando aquellas conexiones que crucen algún obstáculo.
3. Se pone al día la matriz de costes con los nuevos nodos- 4. Se encuentra el camino de menos coste (Dijkstra, A*, etc).
5. Si el algoritmo falla al encontrar la solución, se vuelve a la fase de aprendizaje, generando más puntos si es necesario.

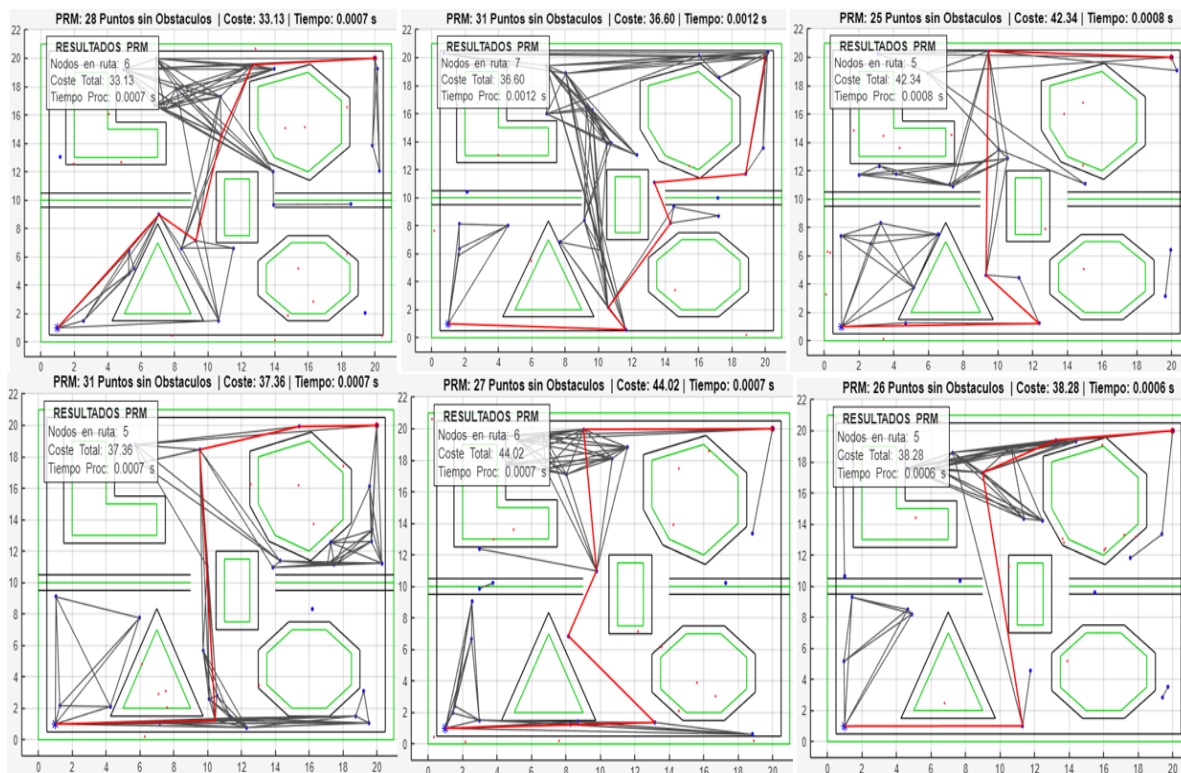


Cabe resaltar que, dado que el método PRM es probabilístico, cada vez que se repite puede dar un resultado diferente. Puede que el camino encontrado no sea el más corto posible. Hasta puede ser que no se encuentre el camino, aunque este exista. Sin embargo, si se repite suficientes veces con suficientes puntos generados, terminará encontrado el camino si este existe. En cuanto a la optimización del camino, a más puntos se generan es mayor la probabilidad de encontrar el camino más corto. Por otra parte, a más puntos, mayor coste computacional.

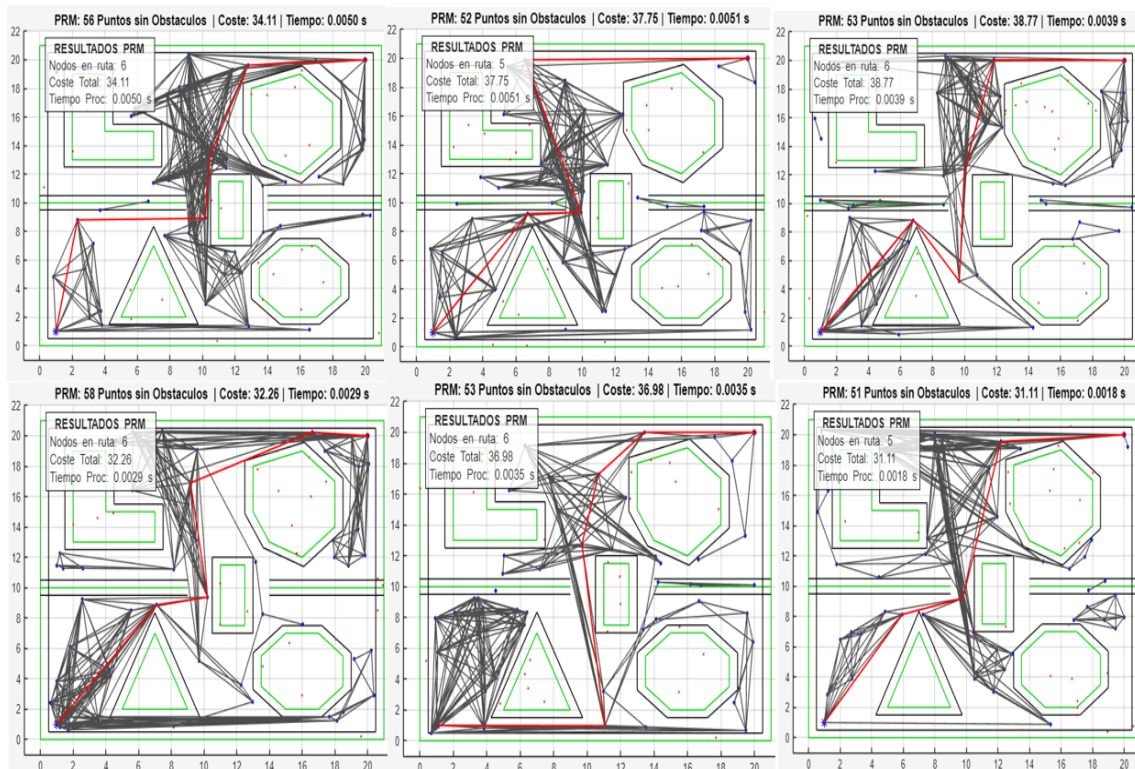
20 puntos



40 puntos



100 puntos



Método RRT (Rapidly exploring random trees- Árboles de exploración rápida aleatoria)

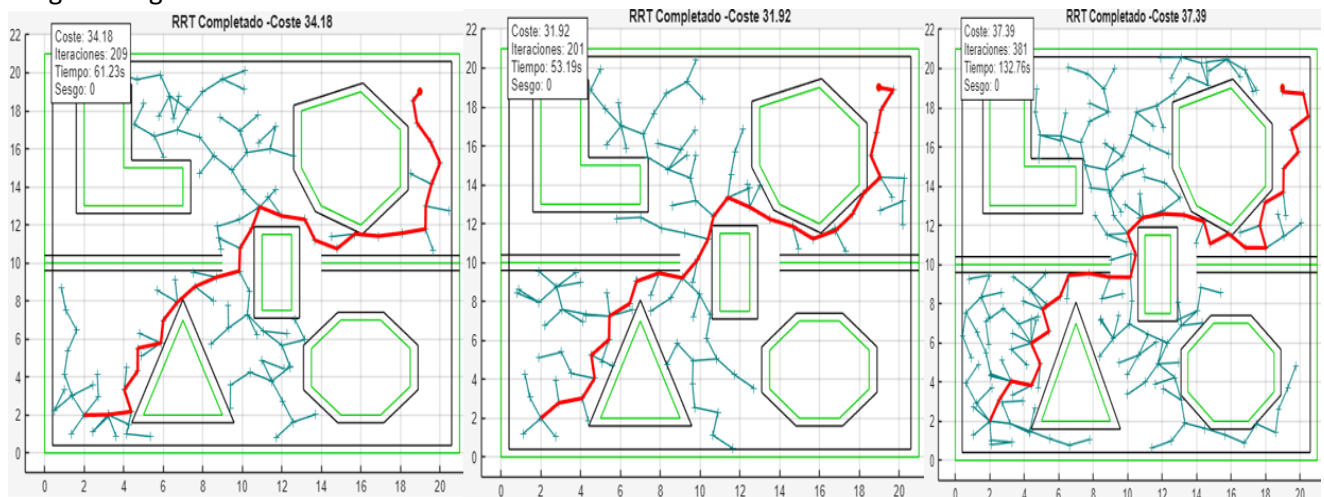
Este algoritmo consiste en generar un árbol de puntos conectados entre ellos. Se parte del punto inicial del robot y se van generando nodos aleatorios. Para cada punto nuevo creado aleatoriamente se busca el nodo más cercano del árbol. Se traza la línea que los une. Se calcula el punto de esta línea que está a una distancia predefinida (en esta práctica un metro) del nodo más cercano del árbol. Si este punto no interseca ningún obstáculo, se añade al árbol.

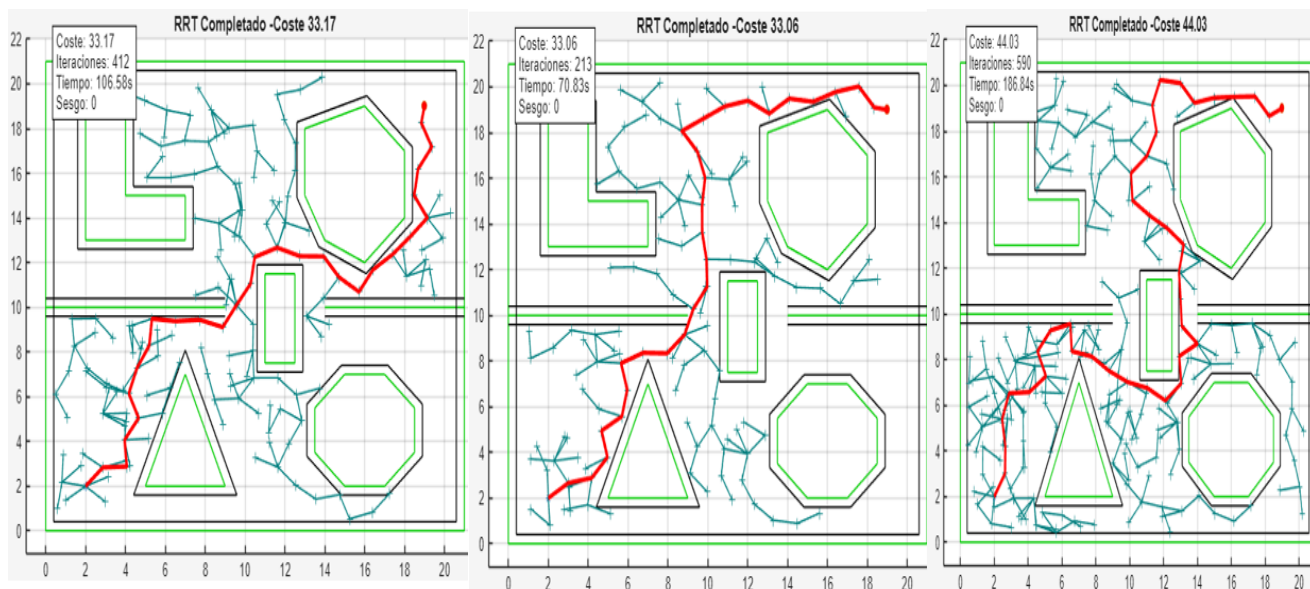
De esta forma, el árbol se expande hasta alcanzar el objetivo.

5.1. RRT básico

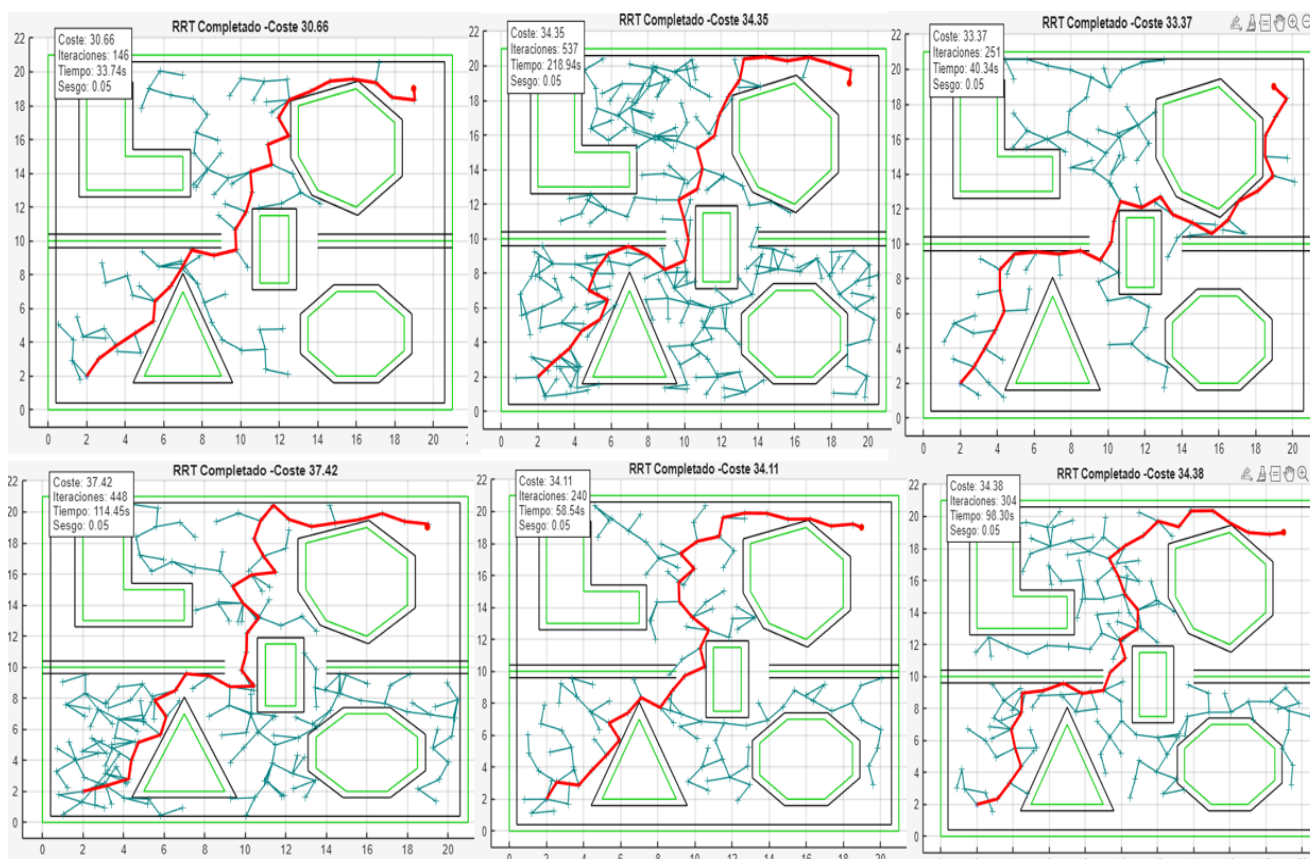
Se trata de un método es aleatorio, por prueba y error. Posiblemente no sea ni el más rápido ni el óptimo, pero siempre encuentra el camino si este existe.

Sesgo 0 Longitud de rama 1

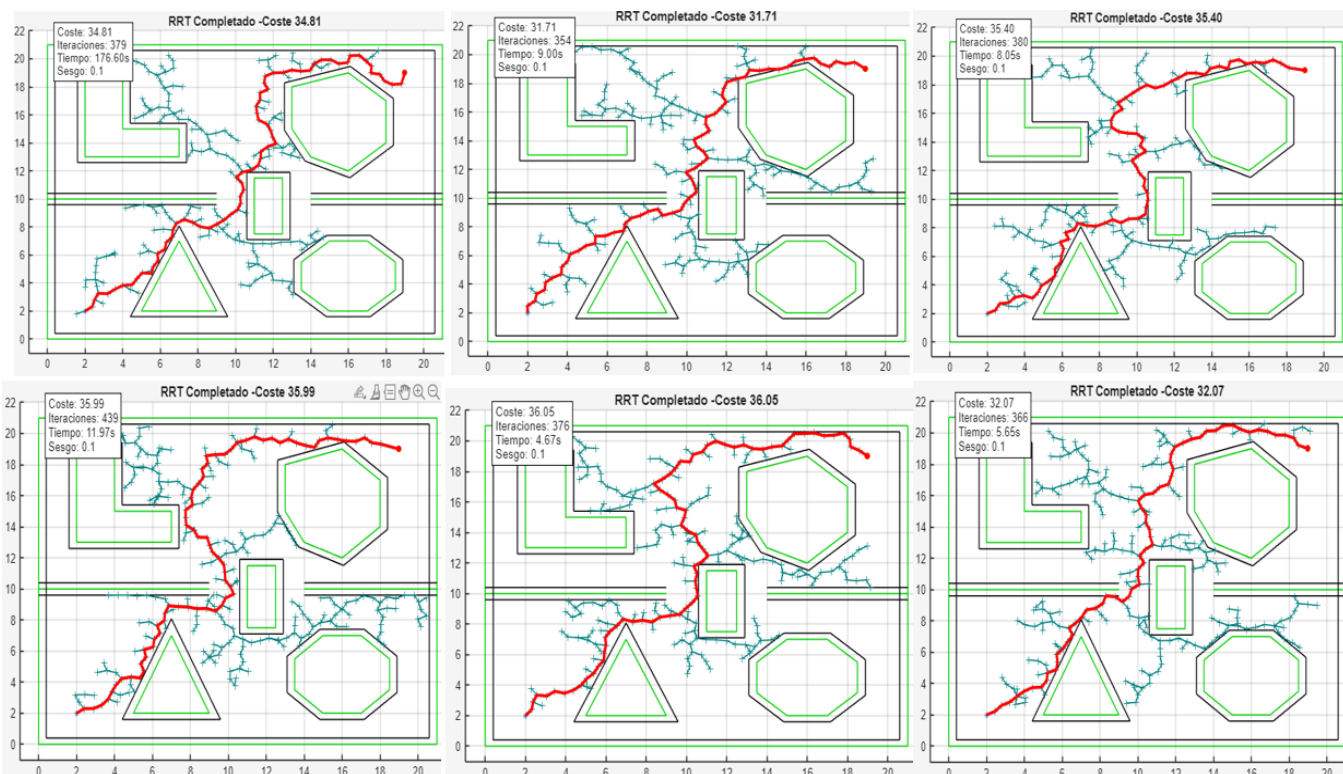




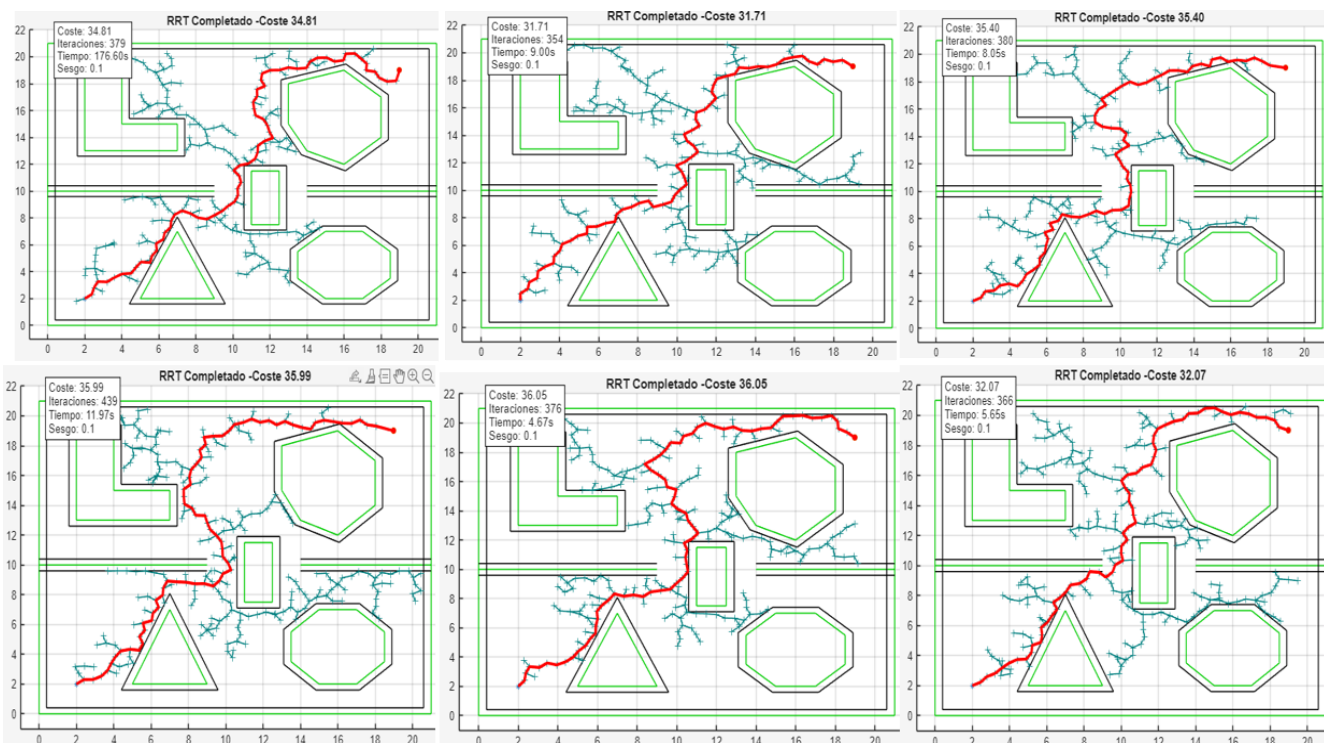
Sesgo 0.05 Longitud de rama 1



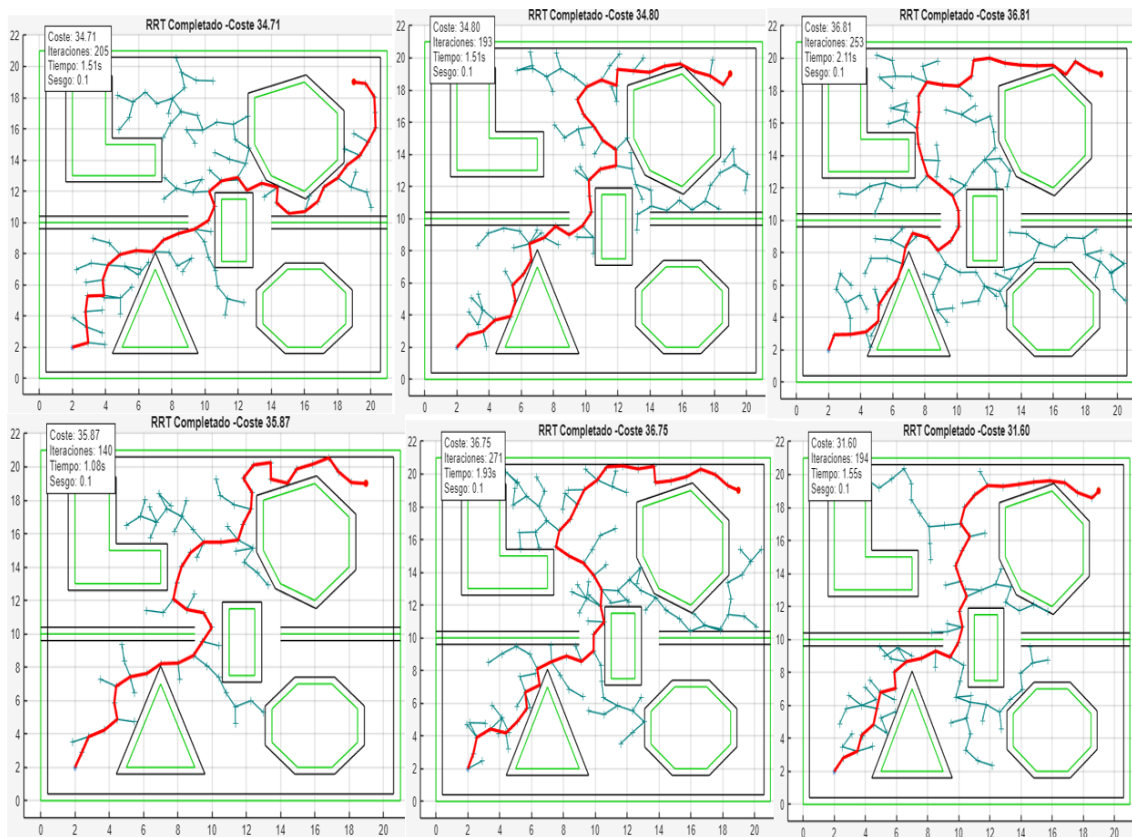
Sesgo 10 % Longitud de rama 1



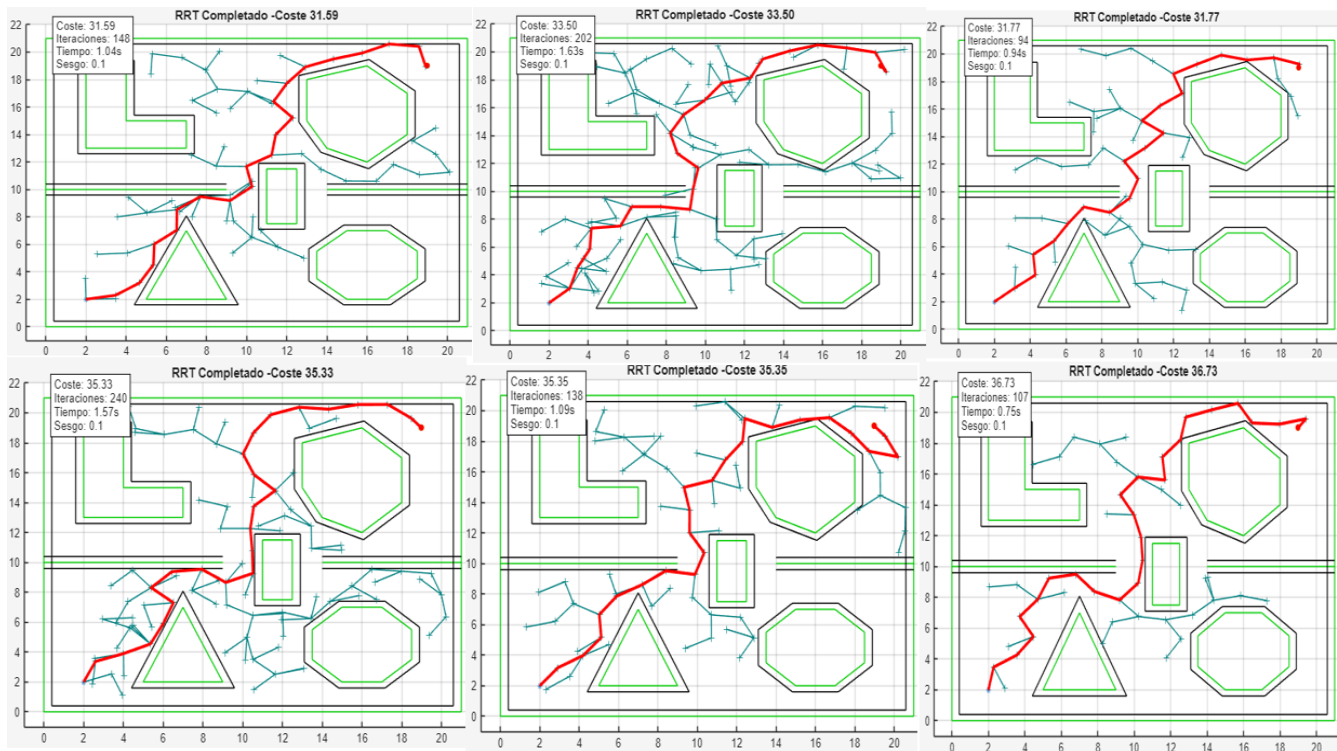
Longitud de 0.5 Sesgo 10 %



Longitud de 1 Sesgo 10 %



Longitud de 1.5 Sesgo 10 %

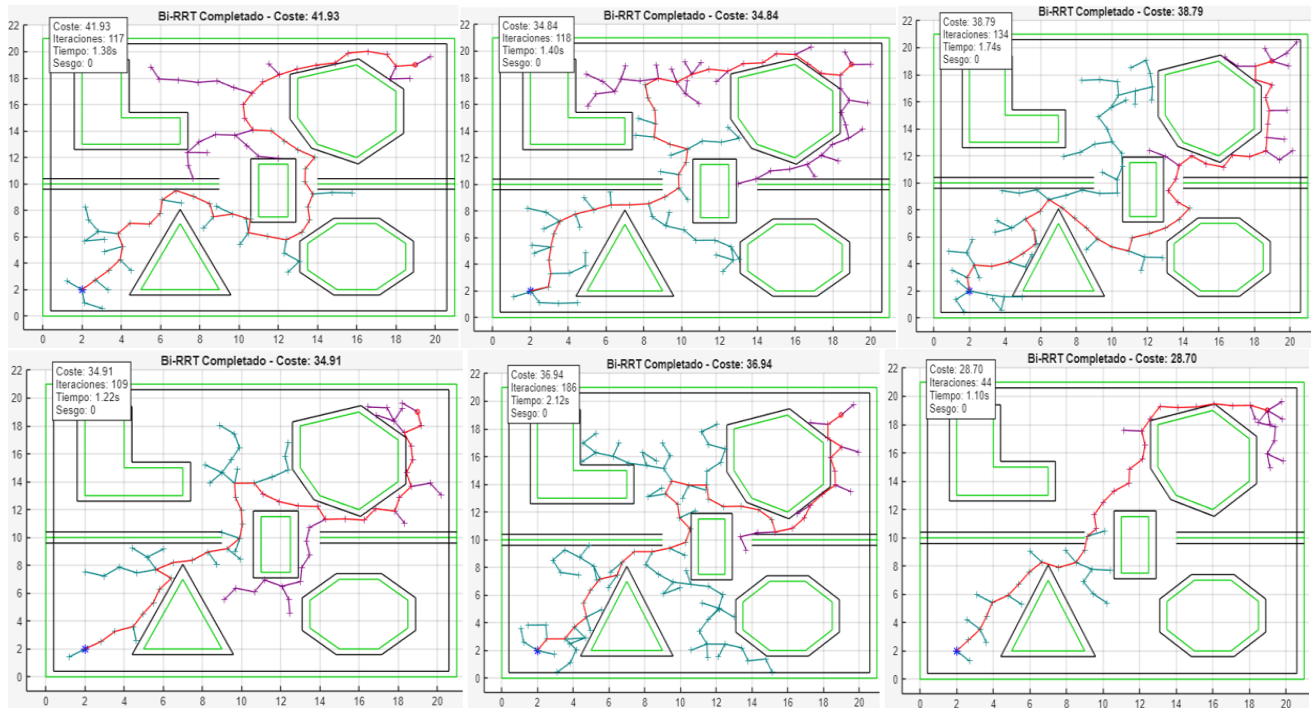


5.2. RRT bidireccional

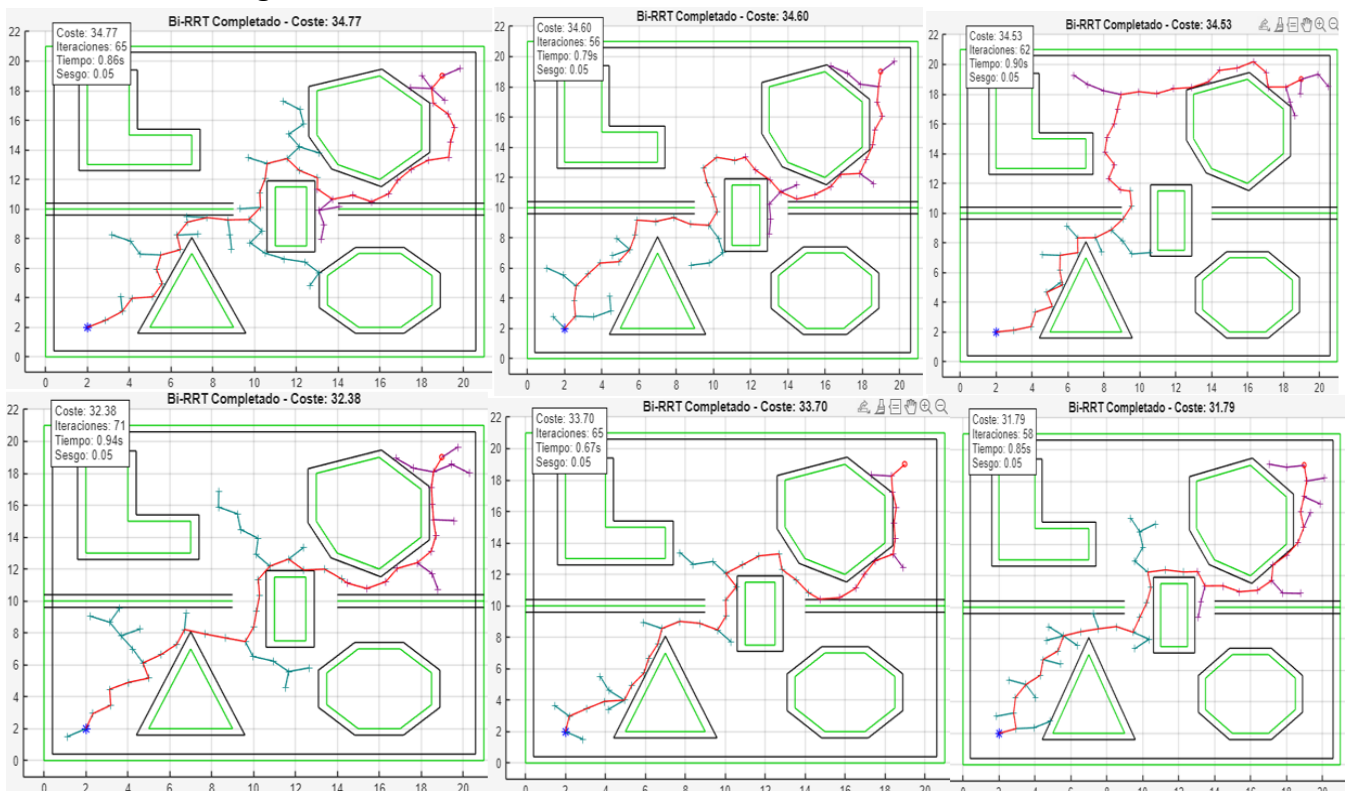
Este método se diferencia del anterior en que se crean dos árboles en lugar de uno. El primero parte desde el punto inicial. El segundo desde el objetivo.

En cada iteración se le añade una nueva rama a cada uno. La búsqueda se termina cuando los árboles se encuentran

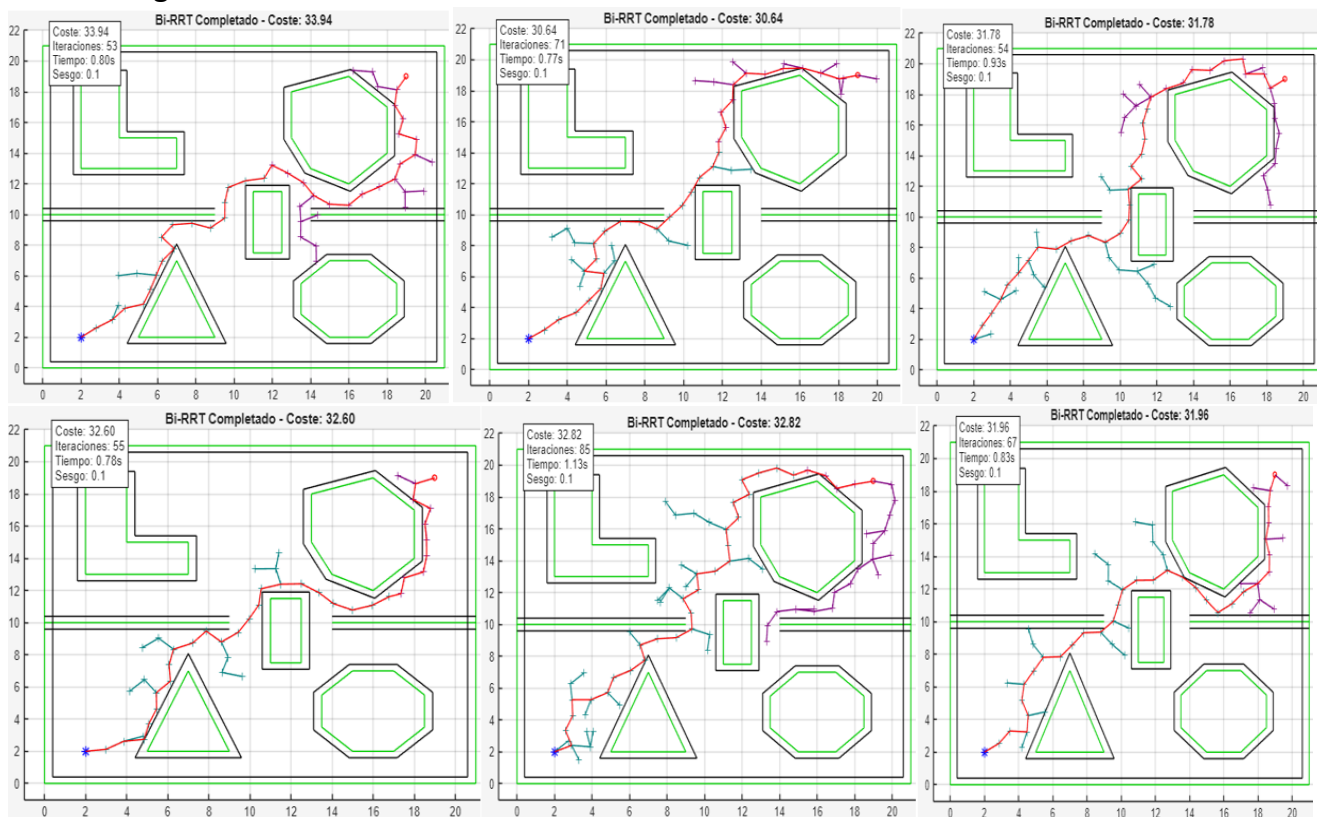
BI-RRT Sesgo 0



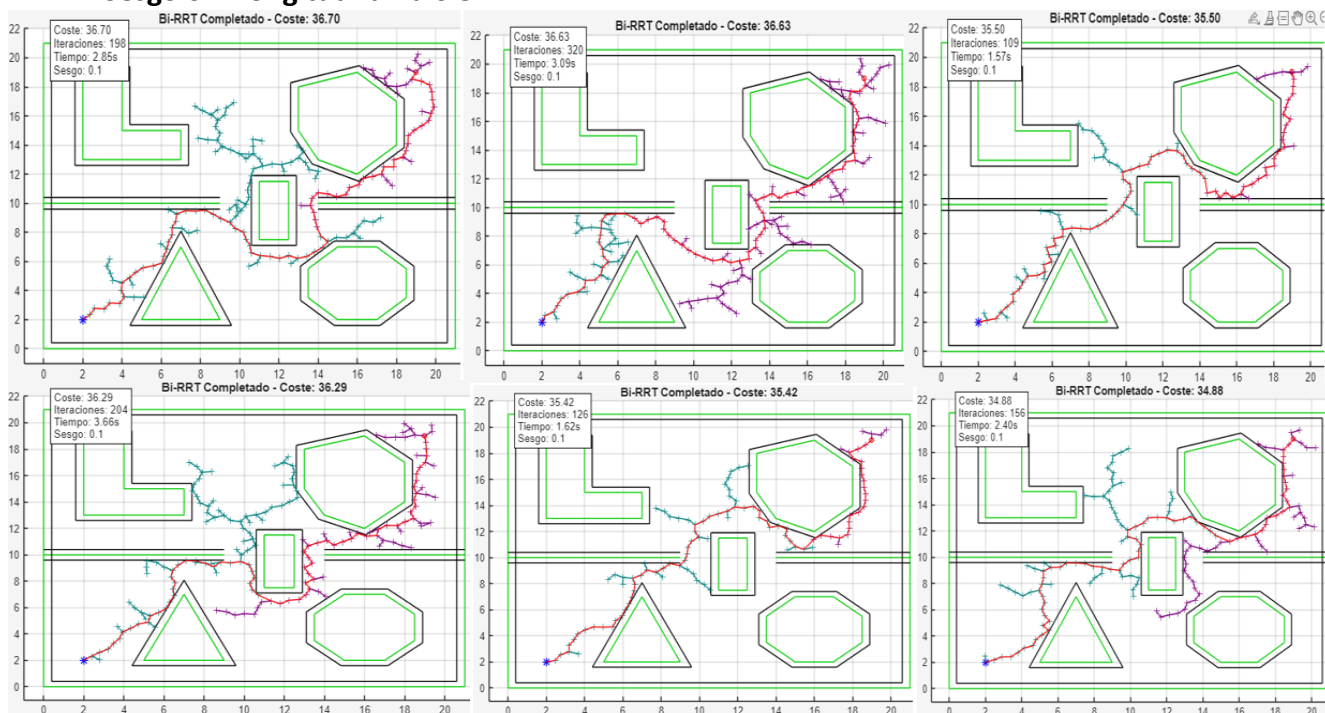
BI-RRT Sesgo 0.05



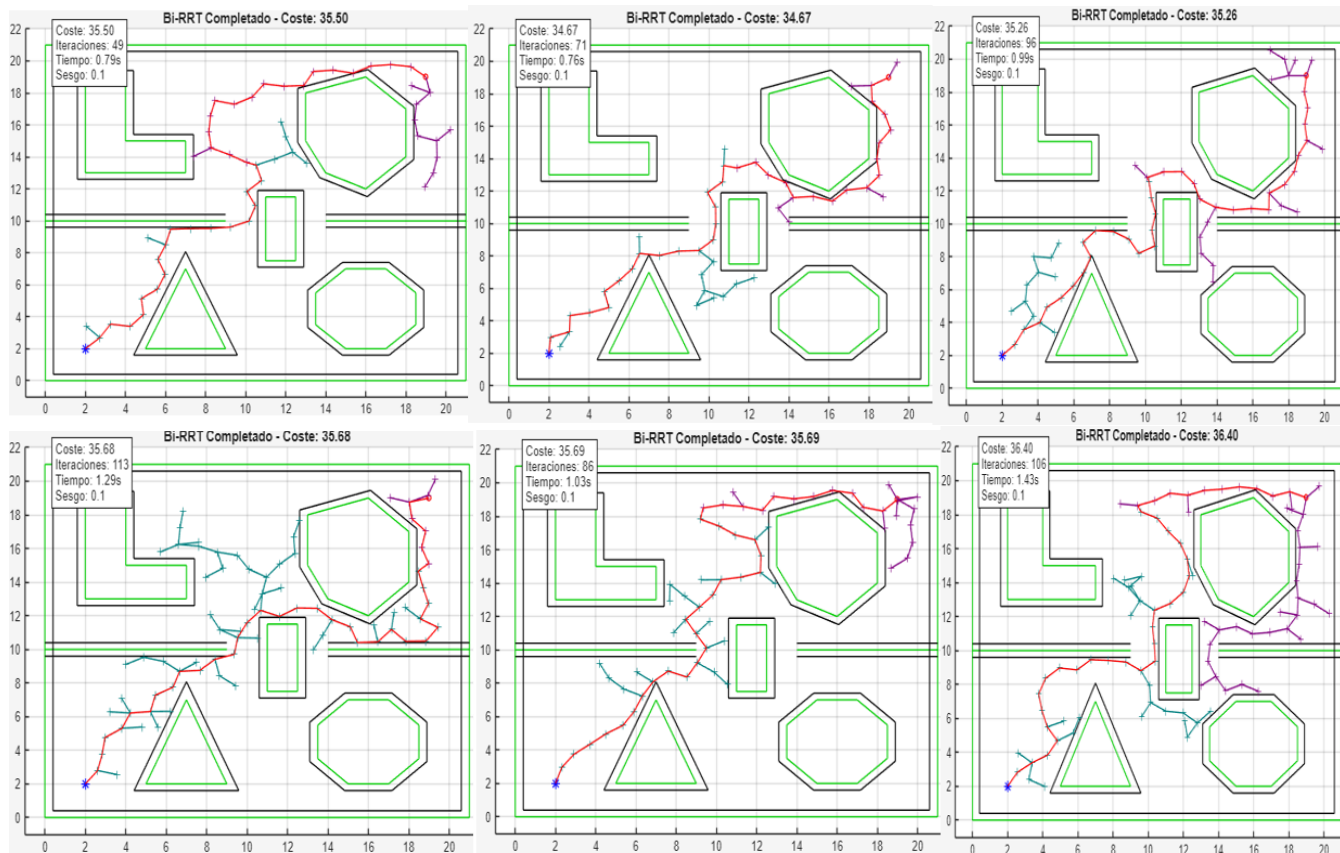
BI-RRT Sesgo 0.1



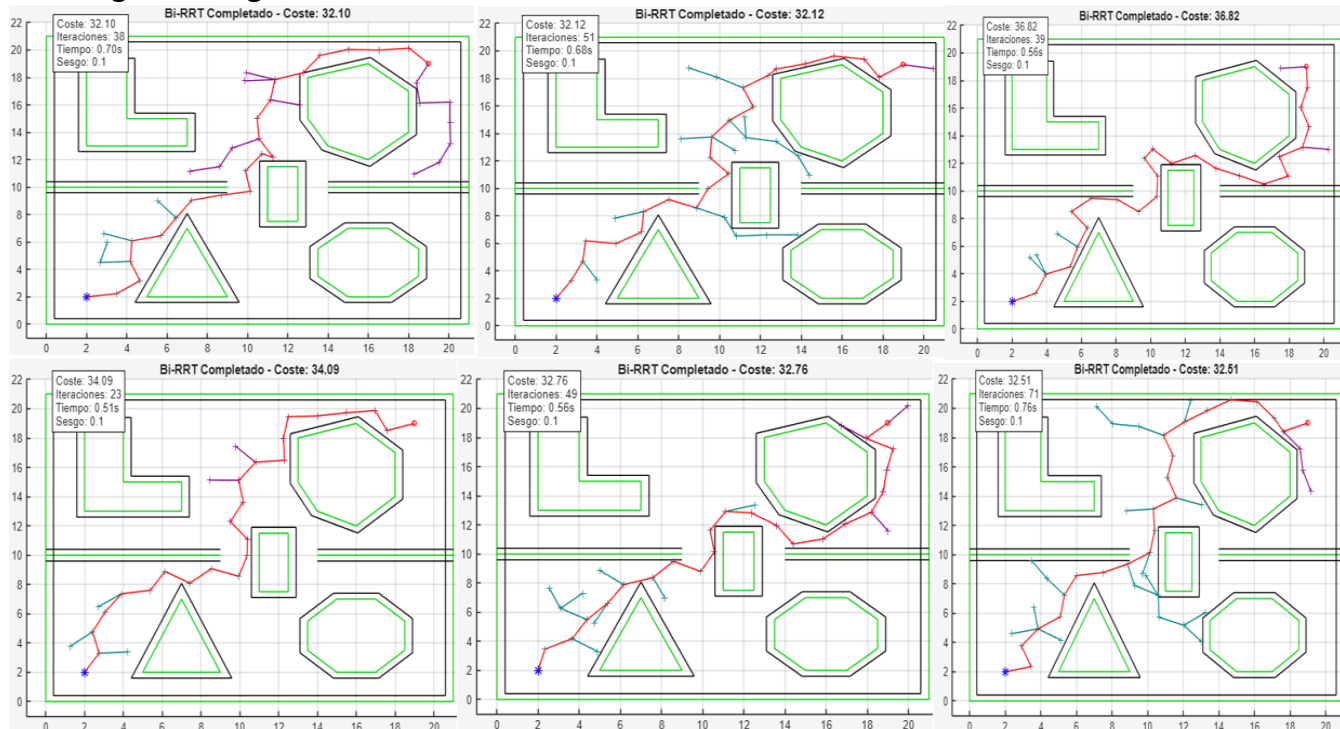
BI-RRT Sesgo 0.1 Longitud rama 0.5



BI-RRT Sesgo 0.1 Longitud rama 1



RRT Sesgo 0.1 Longitud rama 1.5



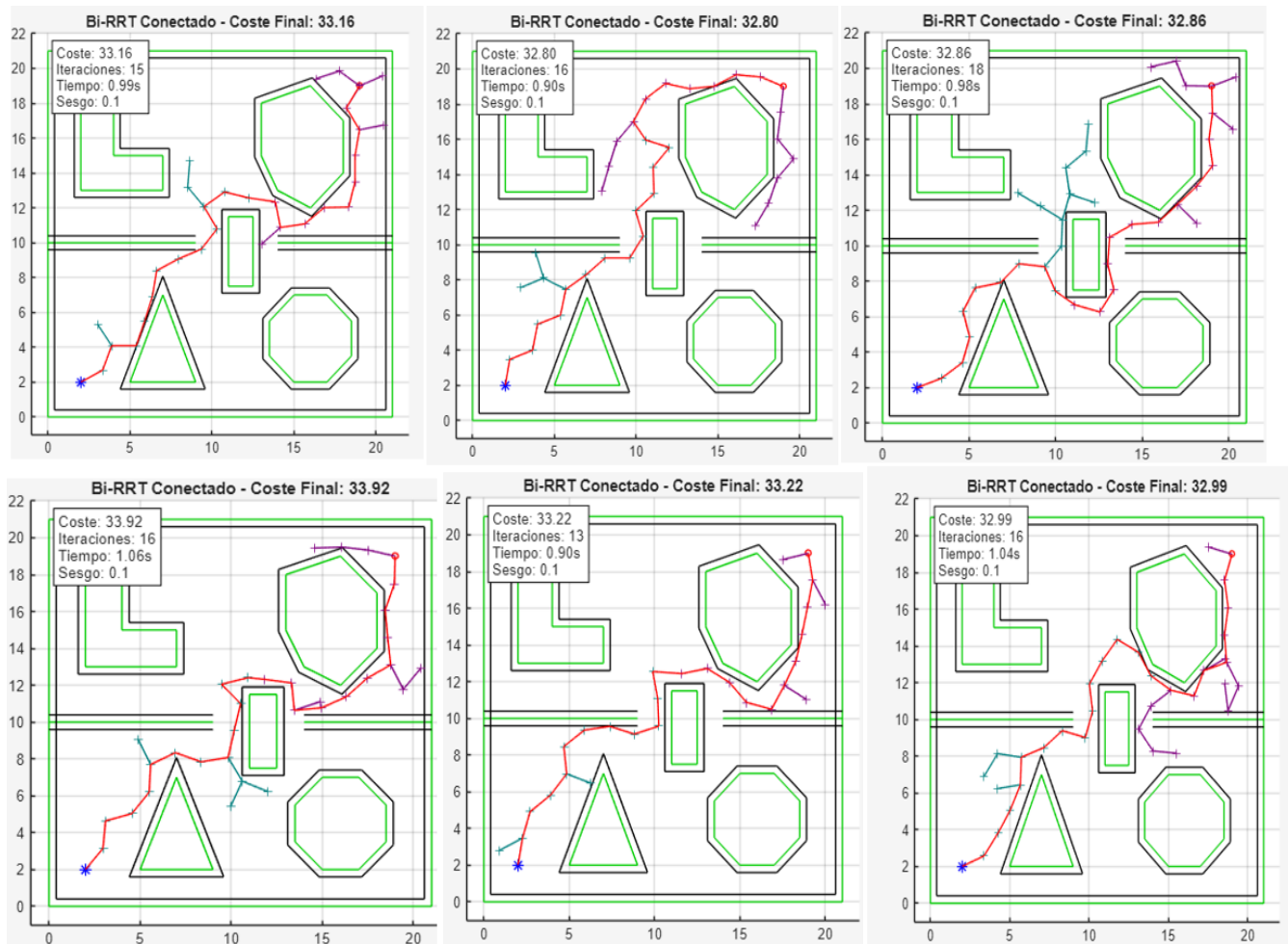
5.3. RRT- Connect

Este algoritmo se implementó para encontrar un camino más corto de forma más rápida, así como para suavizar el camino obtenido por un RRT clásico. introduce unos cambios con respecto al RRT clásico:

Es bidireccional, al menos en la versión original.

Las ramas no se expanden solo una vez, sino o bien hasta tocar un obstáculo, o bien hasta alcanzar el punto aleatorio que generó la rama.

Cada vez que se crea un nuevo nodo se verifica si algún nodo del otro árbol es "visible". Si esto ocurre, se unen los árboles y con esto se termina la búsqueda.



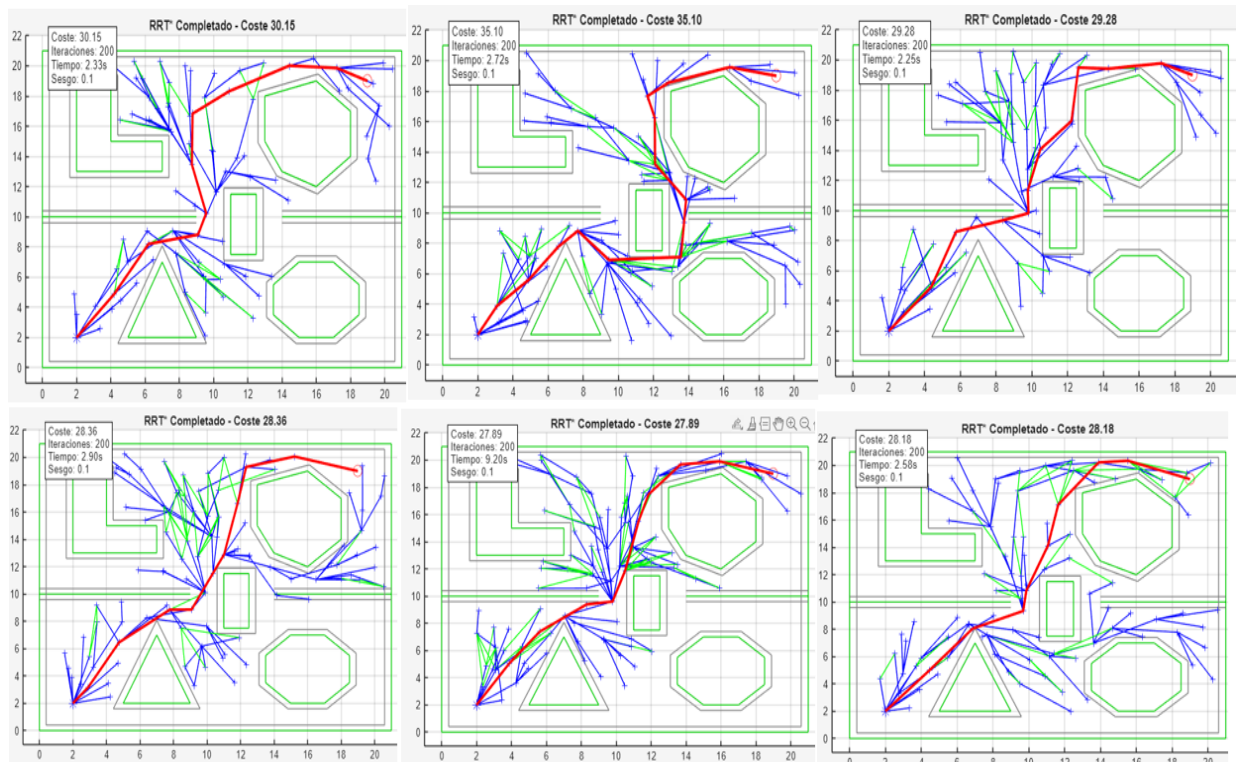
5.4. RRT*

Este algoritmo, al igual que el anterior, se ha desarrollado para mejorar la suavidad y el coste del camino obtenidos por el RRT básico. El usuario ha de especificar otro parámetro adicional: *la vecindad*. Se trata de una distancia.

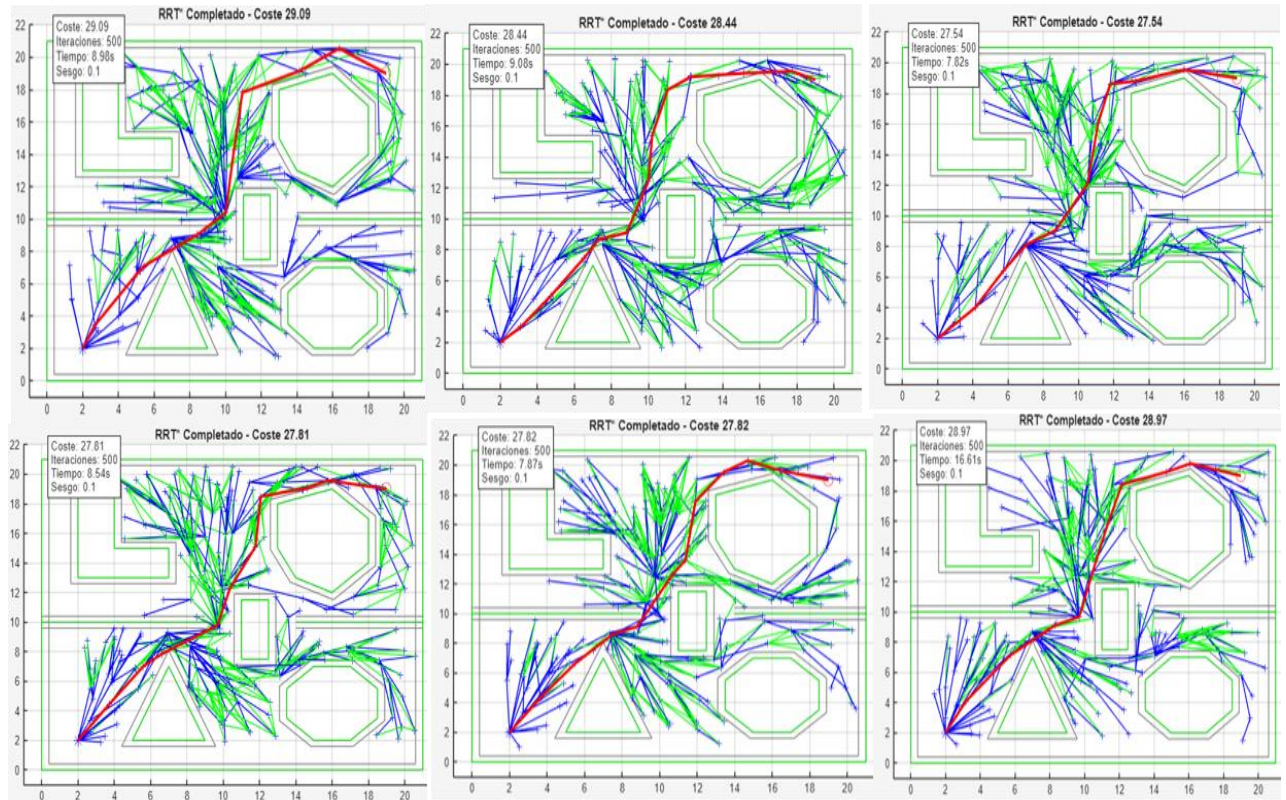
La expansión del árbol en este método se puede dividir en las siguientes fases:

1. Al generar un nuevo nodo aleatorio, al igual que en el RRT básico, se genera una rama desde el nodo más cercano.
2. Se verifica el coste del nuevo desde el inicio pasando por todos los de la vecindad. Se elige el más corto de todos.
3. Se compara el coste de cada nodo de la vecindad a través del nuevo nodo con el valor inicial. Si es menos costos a través del nuevo nodo, se modifica el árbol.

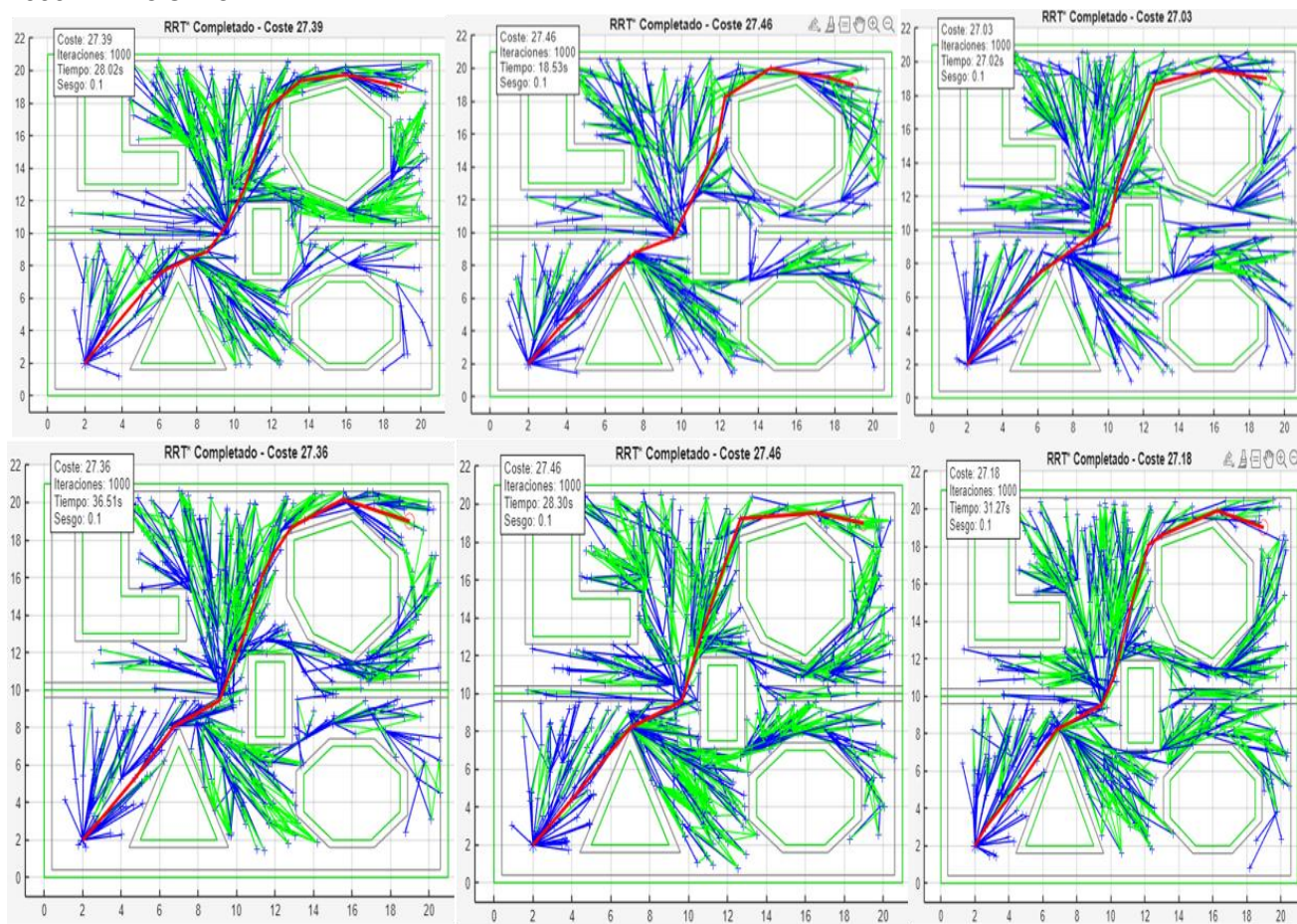
200 ITERACIONES



500 ITERACIONES



1000 ITERACIONES



Tablas

6. Resultados, comparación y conclusiones

6.1 Comparación del algoritmo PRM con diferente número de puntos

Tabla 6.1. Comparación de PRM según el número de puntos

Configuración	T. Med (s)	Var. T	T. Máx (s)	T. Mín (s)	C. Medio	Var. C	C. Máx	C. Mín
20 Puntos	0.0006	4.20E-08	0.0009	0.0004	42.5	25.4	48.2	38
40 Puntos	0.0008	6.50E-08	0.0012	0.0006	38.1	12.3	41.5	35.2
100 Puntos	0.0015	9.80E-08	0.0021	0.0011	35.2	0.45	36	34.5

Observation

Se observa que, al aumentar el número de puntos disponibles en la hoja de ruta probabilística, el coste del camino disminuye progresivamente. Esto se debe a que la conectividad del grafo mejora y aparecen rutas más eficientes. Sin embargo, el tiempo de construcción también aumenta debido al mayor número de conexiones que deben evaluarse.

6.2 Comparación de RRT según el sesgo & Longitud de rama

Tabla 6.2. Comparación de RRT

Algoritmo	T. Med (s)	Var. T	T. Max (s)	T. Min (s)	C. Med	Var. C	C. Max	C. Min
RRT (Sesgo 0%, L 1.0)	0.098	0.0015	0.142	0.055	33.66	1.83	36.2	31.5
RRT (Sesgo 5%, L 1.0)	0.115	0.0043	0.185	0.045	33.3	3.51	37.1	30.5
RRT (Sesgo 10%, L 0.5)	0.045	0.0025	0.11	0.012	35.8	2.1	39.5	32.5
RRT (Sesgo 10%, L 1.0)	0.024	0.0032	0.095	0.005	34.62	2.5	38	31.8
RRT (Sesgo 10%, L 1.5)	0.036	0.0041	0.12	0.009	36.5	3.2	41.2	33.1

Observation

Se observa que el aumento del sesgo hacia el objetivo reduce significativamente el tiempo de ejecución. Con un sesgo del 10 %, el algoritmo obtiene tiempos de búsqueda mucho menores y una menor dispersión en los costes. Además, mantiene una calidad de trayectoria similar a la obtenida con sesgos menores. Por ello, el valor de sesgo del 10 % se selecciona como la mejor configuración para las siguientes comparaciones.

Tabla 6.3 Comparación de BI-RRT

Algoritmo	T. Med (s)	Var. T	T. Max (s)	T. Min (s)	C. Med	Var. C	C. Max	C. Min
BI-RRT (S 0.0, L 1.0)	2.1	0.45	3.5	1.2	35.2	2.1	38.5	32
BI-RRT (S 0.05, L 1.0)	1.5	0.3	2.8	0.9	34.8	2.5	39	31.5
BI-RRT (S 0.1, L 0.5)	1.8	0.5	3	1.1	35.5	2.8	40.2	31.8
BI-RRT (S 0.1, L 1.0)	1.25	0.28	2.1	0.6	34.3	3.2	39.5	30.2
BI-RRT (S 0.1, L 1.5)	1.45	0.35	2.5	0.8	34.9	3.5	41	30.8

Análisis de Resultados (BI-RRT)

- **Influencia del Sesgo:** El sesgo (probabilidad de muestrear el objetivo) reduce significativamente la varianza del tiempo. Un sesgo del **0.1 (10%)** permite que los dos árboles se encuentren en el espacio libre de forma más dirigida, evitando exploraciones innecesarias en zonas alejadas de la meta.
- **Influencia de la Longitud de Rama:**
 - **0.5:** Aunque genera un camino más suave y detallado, aumenta el tiempo de computación al requerir más iteraciones para cubrir la misma distancia.
 - **1.0:** Se muestra como el punto óptimo en este entorno. Proporciona el mejor equilibrio entre el tiempo de expansión y la calidad del coste del camino.
 - **1.5:** Una rama muy larga puede "saltar" espacios estrechos o colisionar con obstáculos con mayor facilidad, lo que incrementa el coste del camino final al generar movimientos menos precisos en zonas complicadas.

Recomendación para BI-RRT

Para optimizar el BI-RRT en tu entorno, la configuración recomendada es **Sesgo 0.1 y Longitud de rama 1.0**. Esta combinación minimiza el tiempo de cálculo (media de 1.25s) y mantiene el coste en niveles muy competitivos. Si el entorno es muy estrecho o complejo, considera reducir la longitud de rama a 0.5 a cambio de un ligero incremento en el tiempo de procesamiento para ganar precisión en el camino.

Tabla 6.4. Comparación global de los mejores algoritmos

Algoritmo	T. Med (s)	Var. T	T. Máx (s)	T. Mín (s)	C. Medio	Var. C	C. Máx	C. Mín
RRT Básico	0.024	0.0032	0.095	0.0052	34.62	2.5	38	31.8
BI-RRT	1.25	0.28	2.1	0.6	34.3	3.2	39.5	30.2
RRT-Connect	0.85	0.05	1.1	0.5	33.5	1.1	35.8	31.2
RRT (200)*	4.5	0.8	6.2	3.1	32.8	0.5	33.5	32.1
RRT (500)*	10.2	2.1	13.5	7.2	31.5	0.25	32	30.8
RRT (1000)*	22.1	4.5	28	16.5	30.2	0.1	30.5	29.8

6.5 Discusión de resultados

Los resultados obtenidos muestran claramente las diferencias entre las distintas técnicas de planificación de trayectorias estudiadas.

El algoritmo PRM presenta tiempos de cálculo extremadamente bajos gracias a que la búsqueda se realiza sobre una hoja de ruta previamente construida. Sin embargo, la calidad de las trayectorias depende directamente del número de puntos utilizados.

El algoritmo RRT básico consigue encontrar soluciones válidas en todos los casos, aunque presenta una elevada dispersión tanto en el coste como en el tiempo de ejecución. La incorporación de un sesgo hacia el objetivo mejora considerablemente su comportamiento.

BI-RRT mejora significativamente el rendimiento del RRT clásico mediante el crecimiento simultáneo de dos árboles, uno desde el punto inicial y otro desde el objetivo. Esto reduce el número de iteraciones necesarias para encontrar una conexión válida.

RRT-Connect proporciona una mejora adicional al extender los árboles de forma continua hasta encontrar obstáculos o conectar ambos extremos. Gracias a ello obtiene trayectorias más cortas y tiempos de ejecución reducidos.

Finalmente, RRT* incorpora un proceso de optimización mediante reconexión de nodos. Esto permite mejorar progresivamente la calidad de la trayectoria obtenida y converger hacia soluciones cercanas al óptimo. Sin embargo, esta mejora implica un aumento significativo del tiempo de cálculo, especialmente cuando se incrementa el número de iteraciones.

6.6 Conclusiones

Tras realizar todas las pruebas experimentales puede concluirse que:

- El aumento del número de puntos en PRM mejora la calidad del camino, aunque incrementa el tiempo de construcción del grafo.
- Un sesgo del 10 % proporciona los mejores resultados para RRT, reduciendo notablemente los tiempos de búsqueda.
- Las longitudes de rama mayores permiten explorar el entorno más rápidamente y obtener soluciones en menos iteraciones.
- BI-RRT ofrece una mejora clara respecto al RRT clásico tanto en coste como en tiempo de ejecución.
- RRT-Connect proporciona una excelente relación entre rapidez y calidad de trayectoria.
- RRT* obtiene las trayectorias de menor coste entre todos los algoritmos analizados.

Por tanto, para este entorno de simulación, **RRT*** es el algoritmo que genera las mejores trayectorias, mientras que **BI-RRT** y **RRT-Connect** ofrecen el mejor compromiso entre calidad de la solución y tiempo de cálculo.

En términos de calidad de trayectoria, el mejor algoritmo ha sido RRT con 1000 iteraciones. En términos de eficiencia computacional, los mejores resultados se obtuvieron con BI-RRT y RRT-Connect.*